I/O Systems

9th ed: Ch. 13

10th ed: Ch. 12

Objectives

- To understand the general structure of the I/O subsystem
- To know different ways of performing I/O including polling, interrupts, and direct memory access
- To know of different types of device
- To be aware of other issues including caching, scheduling, and performance

Outline

- I/O subsystem
- I/O devices
- Kernel data structures

Outline

- I/O subsystem
  - Polling
  - Interrupts
  - Interrupt handling
    - Direct Memory Access (DMA)
- I/O devices
- Kernel data structures

Computers and computation rely on I/O

- Need input data to process, and means to output results
- There is a huge range of I/O devices
  - **Human readable:** graphical displays, keyboard, mouse, printers
  - **Machine readable:** disks, tapes, CD, sensors
  - **Communications:** modems, network interfaces, radios
- All differ significantly from one another in several ways:
  - **Data rate:** orders of magnitude different between keyboard and network
  - **Control complexity:** printers much simpler than disks
  - **Transfer unit and direction:** blocks vs characters vs frame stores
  - **Data representation:** Mouse sends relative movements (e.g., “move right 5 pixels”), Disk stores raw binary blocks (0s and 1s)
  - **Error handling**
- I/O management is therefore a major component of an OS
  - New devices come along frequently
  - I/O performance is critical to system performance
  - Also wish to present a homogenous API

I/O subsystem

- Incredible variety of I/O devices but there are commonalities
  - Signals from I/O devices interface with computer
  - A device has at least one connection point, or **port**
  - Devices interconnect via a **bus,** either daisy-chained or shared direct access
  - Devices have integrated or separate controllers (host adapters) containing processor,

microcode, private memory, etc that operate the device, handle bus connections, any ports

- Typically device will have registers to hold commands, addresses, data
  - E.g., Data-in register, data-out register, status register, control register

. . . .

. . . .

I/O operations

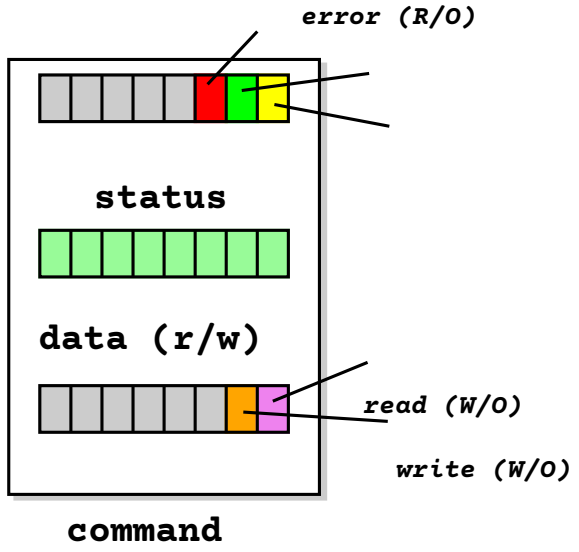
Enable communication between the computer system and the external environment. The operating system manages these devices to ensure smooth data transfer between hardware and software.

Categories of I/O Operations

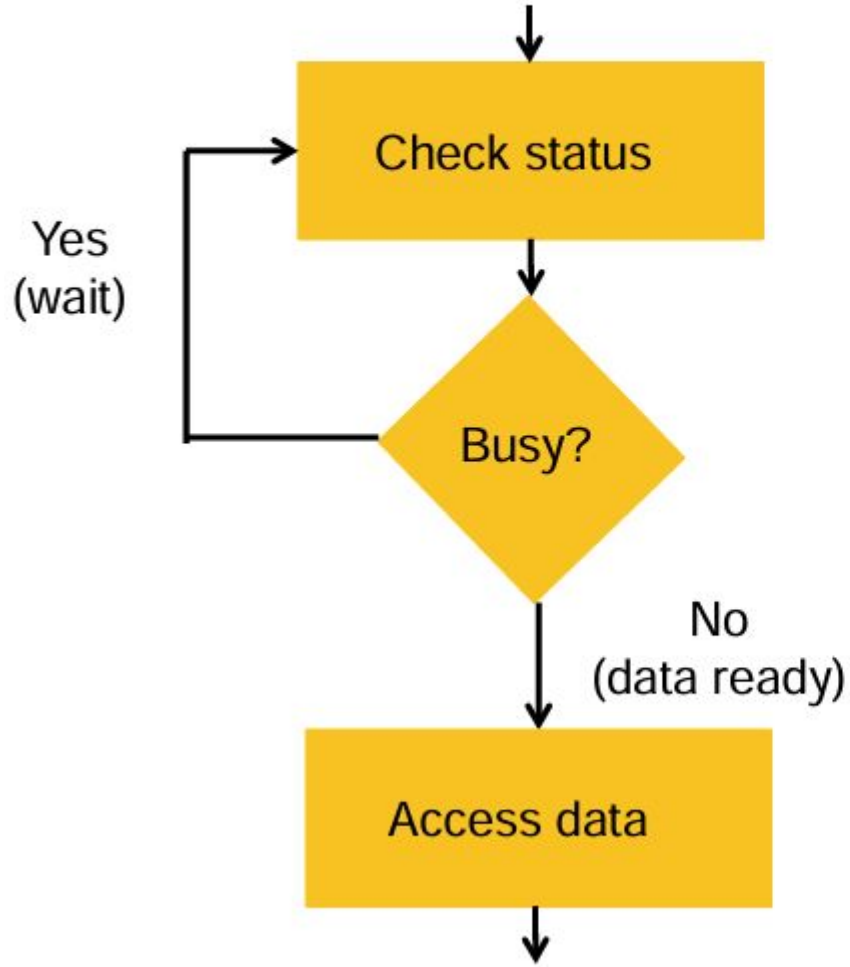
- **Programmed I/O/Polling:** CPU directly controls all I/O operations — simple but inefficient.
- **Interrupt-Driven I/O:** CPU performs other tasks and is interrupted only when I/O is ready.
- **Direct Memory Access (DMA):** Allows devices to transfer data directly to/from memory without CPU intervention — improves performance.

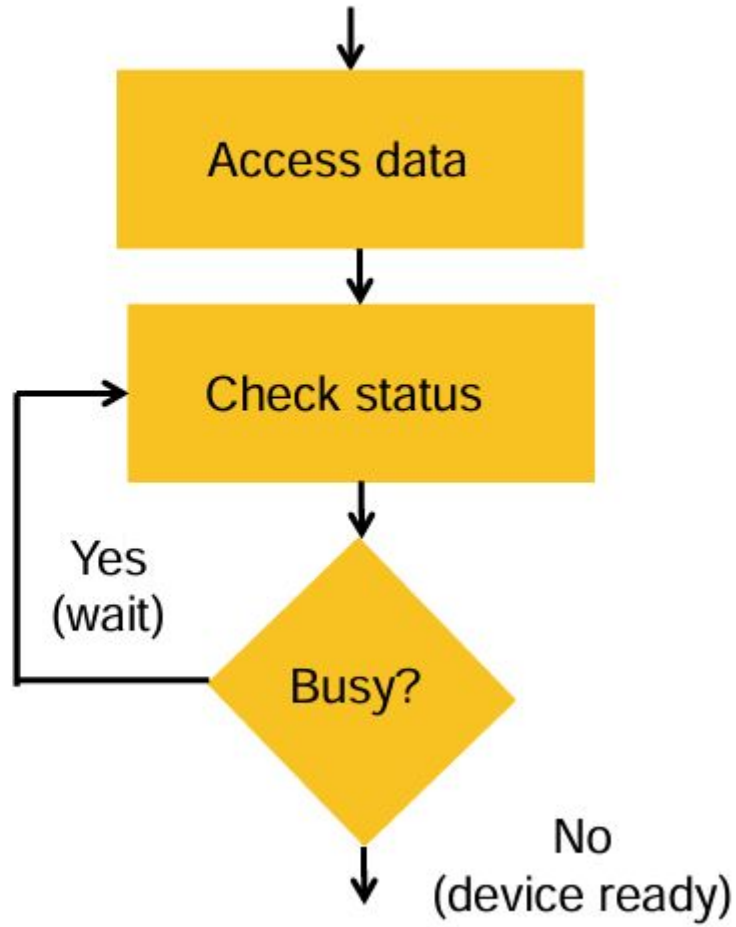
Polling

- Consider a simple device
  - Three registers: status, data and command
  - Host can read and write registers via the bus
- Polled mode operation is as follows, for every byte:
  - Host repeatedly reads *device-busy* until clear
  - Host sets *read* or *write* bit in command register, and puts data into data register
  - Host sets *command-ready* bit in status register
  - Device sees *command-ready* and sets *device-busy*
  - Device performs requested operation, executing transfer
  - Device clears *command-ready* and any *error* bit, and then clears *device-busy*
- Step 1 is polling – a **busy-wait** cycle, waiting for some I/O from device
  - This is ok if the device is fast but very inefficient if not
  - If the CPU switches to another task it risks missing a cycle leading to data being overwritten or lost



Busy-wait I/O (“program-controlled”)





I/O operations

1. Programmed I/O (Polling I/O)

How It Works:

In Programmed I/O, the CPU directly controls every aspect of the I/O operation. It repeatedly checks (or *polls*) the status of the I/O device to see if it is ready to send or receive data. Once the device is ready, the CPU transfers the data between the device and memory.

Example:

Imagine you're waiting for a pizza delivery:

- Instead of doing anything else, you stand by the door every second, checking if the delivery person has arrived.
- Only when they arrive do you take the pizza.
- While waiting, you do nothing else.

I/O operations

1. Programmed I/O (Polling I/O)

Advantages:

- Simple to implement—no complex hardware needed.
- Good for very slow or simple devices (e.g., keyboard in early systems).

Disadvantages:

- Wastes CPU cycles—CPU is idle while polling.
- Inefficient for high-speed or bursty I/O.
- Poor system throughput—CPU can't do useful work during I/O.

```
while (device_status != READY);  // CPU keeps polling
data = read_from_device();       // CPU reads data
write_to_memory(data);
```

I/O operations

2. Interrupt-Driven I/O

How It Works:

Here, the CPU initiates an I/O request and then goes on to do other tasks. When the I/O device is ready (e.g., data is available or buffer is free), it sends an interrupt signal to the CPU. The CPU then pauses its current work, saves its state, and runs an Interrupt Service Routine (ISR) to handle the I/O. Data is still transferred via the CPU, but only when needed.

Example:

- You order the pizza, then go back to watching a movie.
- When the delivery person arrives, they ring the doorbell (interrupt).
- You pause the movie, get the pizza, then resume watching.

Simplified Flow:

- CPU starts I/O
- CPU continues other work
- Device finishes → sends interrupt

Interrupts

- More efficient than polling when device is relatively infrequently accessed
- Device triggers **interrupt-request line**
  - Checked by the CPU after each instruction
  - Aligns interrupts with instruction boundaries
- **Interrupt handler** receives the interrupt
- **Interrupt vector** dispatches interrupt to correct handler
  - Context switch required before and after
  - Priorities applied

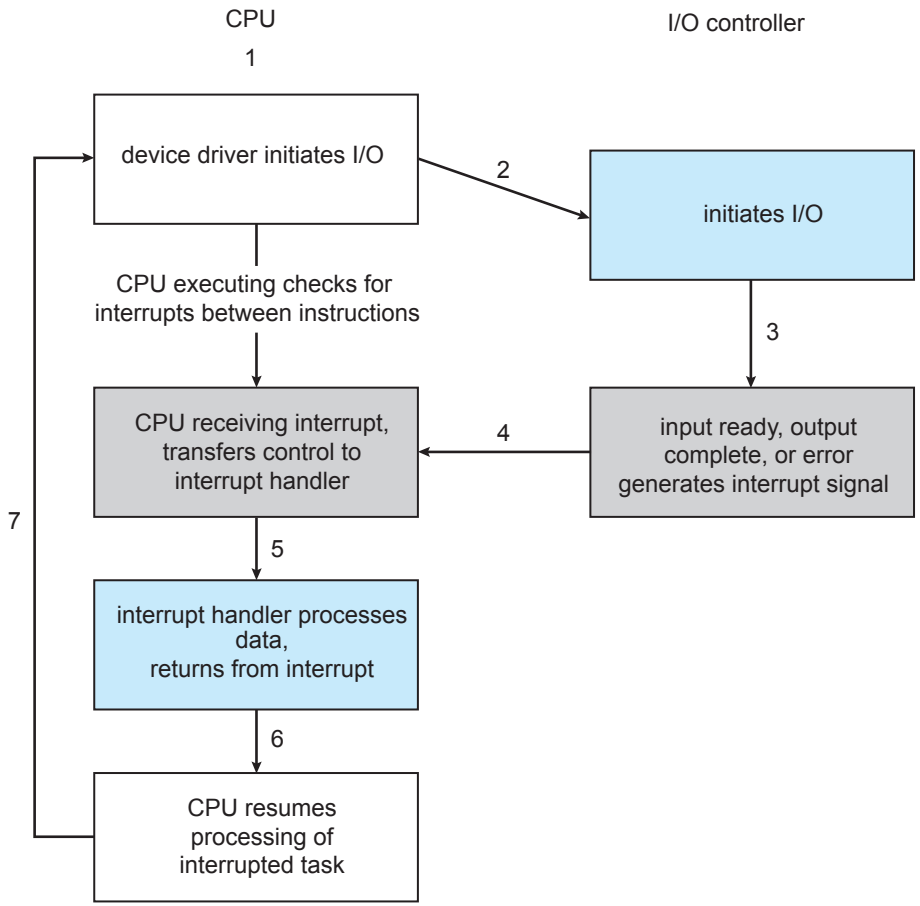


Figure 12.3 Interrupt-driven I/O cycle.

Intel Pentium interrupt vectors

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Figure 12.5 Intel Pentium processor event-vector table.

I/O operations

2. Interrupt-Driven I/O

Advantages:

- Much more efficient—CPU isn't wasted polling.
- Better system responsiveness and multitasking.

Disadvantages:

- Still involves CPU for every data transfer
- Overhead from frequent interrupts (e.g., for high-speed devices like disks or network cards, this causes interrupt storms).
- Latency in responding to interrupts.

I/O operations

3. Direct Memory Access (DMA)

How It Works:

DMA uses a dedicated hardware controller (the DMA controller) to transfer data directly between I/O devices and main memory, without CPU involvement during the actual data transfer.

The CPU only: Initializes the DMA transfer (by providing source/destination addresses and byte count). Gets interrupted when the transfer is complete.

During the transfer, the CPU is free to execute other instructions.

Example: Pizza delivery with a butler

You tell your butler: “When pizza arrives, put it in the fridge.”

You go to work.

The butler handles everything—takes pizza, puts it away.

Only when done does he notify you (optional).

I/O operations

3. Direct Memory Access (DMA)

Advantages:

Massive performance gain—no CPU overhead during bulk transfers.

Ideal for high-speed, large-data devices: disk drives, network cards, video cards.

Reduces interrupt overhead (only 1 interrupt per transfer vs. per byte).

Disadvantages:

Requires extra hardware (DMA controller).

Bus contention: DMA and CPU may compete for memory bus access.

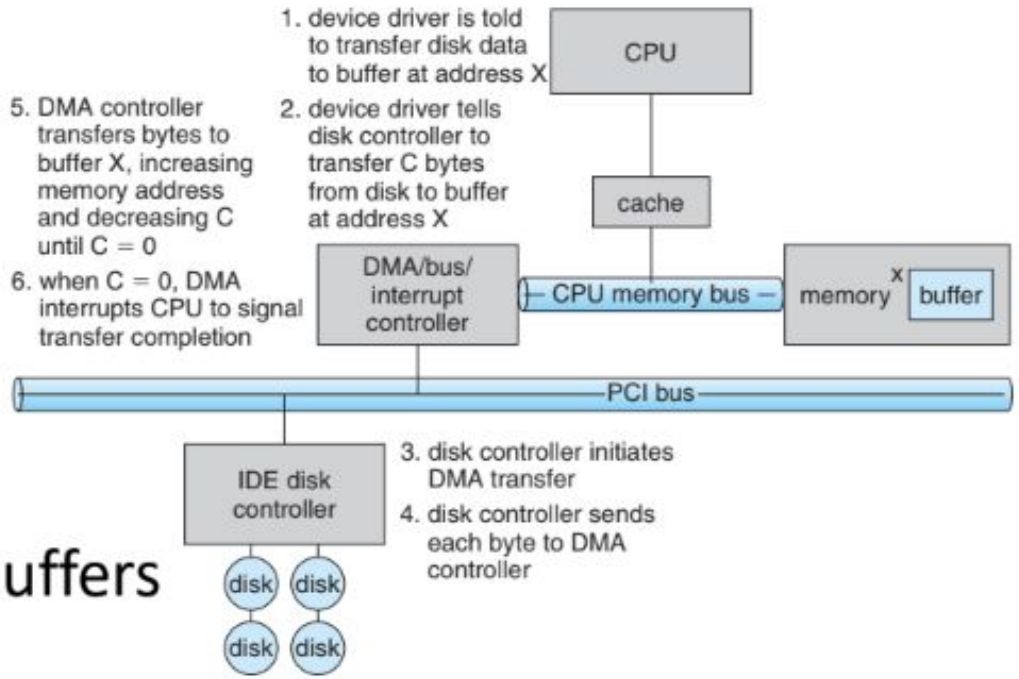
More complex to implement.

Direct Memory Access (DMA)

- Used for high-speed I/O devices able to transmit information at close to memory speeds
  - Interrupts good but (e.g.) **livelock** a problem
  - Better if devices can read and write processor memory **directly** – Direct Memory Access (DMA)
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention with generic DMA "command" include, e.g.,
  - Source address
  - Sink address
  - Transfer size

Direct Memory Access (DMA)

- Only generate one interrupt per block rather than one per byte
- DMA channels may be provided by dedicated DMA controller, or by devices themselves
  - E.g. disk controller passes disk address, memory address and size, and read/write
- All that's required is that a device can become a bus master
  - Requires ability for arbitration as not just CPU driving the bus
  - Involves **cycle stealing** as taking the bus away from the CPU
- Scatter/Gather DMA chains multiple requests, e.g., of disk reads into set of buffers



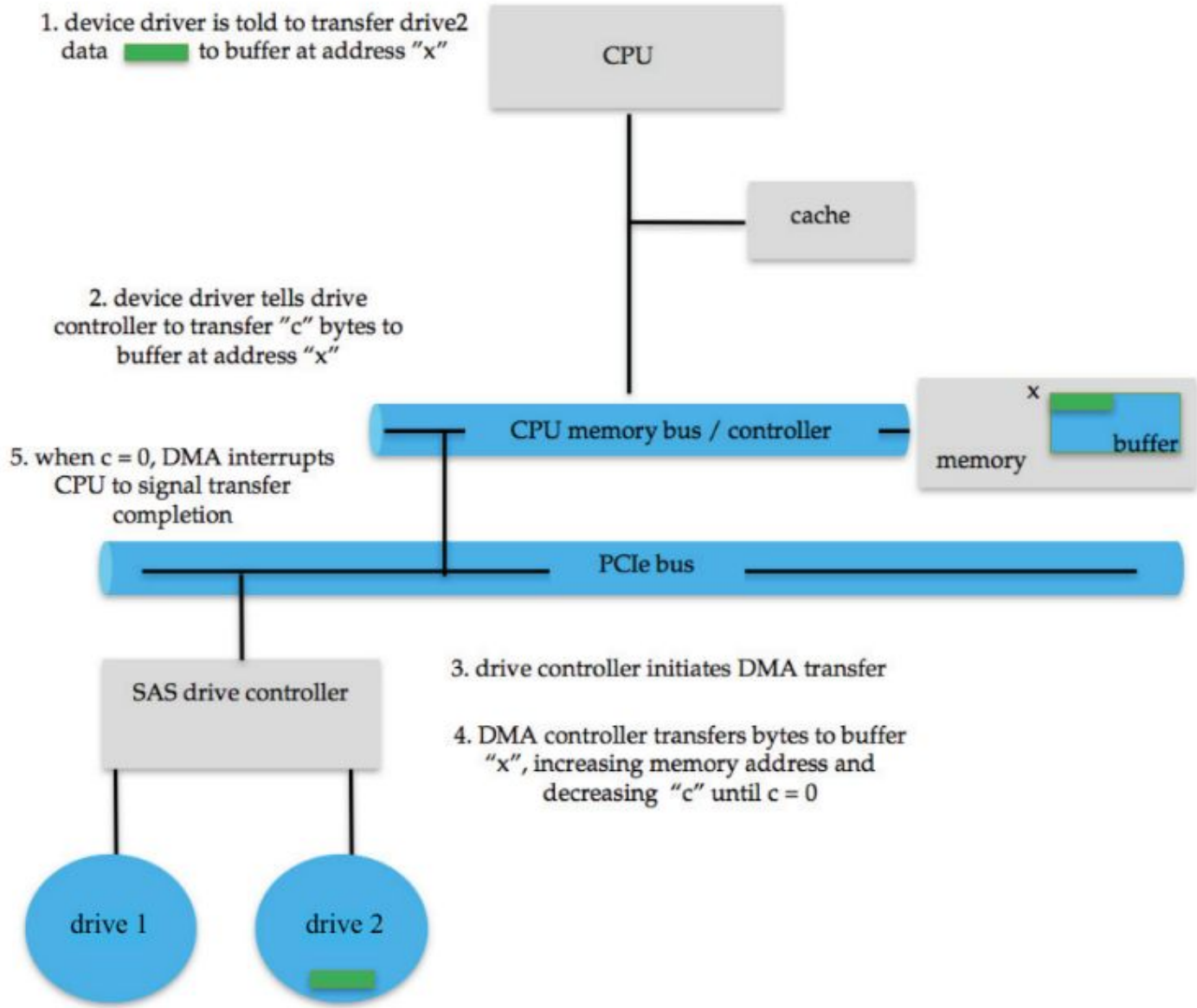


Figure 12.6 Steps in a DMA transfer.

Step-by-Step: How DMA Works

Step 1: Application Requests I/O

- A program asks to read a file → OS tells the disk driver to read data into a memory buffer.

Step 2: Device driver tells device controller

- A program asks to read a file → OS tells the disk driver to read data into a memory buffer.

Step 3: Controller Sets Up DMA Controller

The driver tells the DMA controller:

- Source address: Where on the disk to read from
- Destination address: Which memory buffer to write to
- Transfer size: How many bytes to move

Step 4: DMA Controller Takes Over

- The DMA controller becomes a bus master (temporarily takes control of the memory bus).
- It reads data directly from the disk and writes it straight into RAM — no CPU involvement.

Step 5: CPU Does Other Work

- While DMA is transferring data, the CPU is free to run other programs!

Step 6: Transfer Completes → Interrupt

- When all bytes are transferred, the DMA controller sends one interrupt to the CPU.
- The CPU then wakes up the waiting process and delivers the data.

Outline

- I/O subsystem
- I/O devices
  - Device characteristics
    - Blocking, non-blocking, asynchronous I/O
  - I/O structure
- Kernel data structures

I/O device characteristics

- **Block devices,** e.g. disk drives, CD
  - Commands include *read*, *write*, *seek*
  - (“ Can have *raw* access or via (e.g.) filesystem cooked”) or *memory-mapped*
- **Character devices,** e.g. keyboards, mice, serial
  - Commands include *get*, *put*
- **Network Devices**
  - Vary enough from block and character devices to get their own interface
- **Miscellaneous**
  - Current time, elapsed time, timers, clocks

Aspect	Variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

Figure 12.7 Characteristics of I/O devices.

Speed of operation. Device speeds range from a few bytes per second to a few gigabytes per second.

Read–write, read only, or write only. Some Devices Perform Both Input

Blocking, non-blocking

Blocking I/O

- When a process makes an I/O request (e.g., `read()`), it is suspended (blocked) immediately.
- The OS does not return control to the process until the I/O operation completely finishes.
- During this time, the process cannot do any other work
- Example: A program calls `read()` from a file → waits idle until all data is loaded → then continues.

Non-Blocking I/O

- The I/O system call returns immediately, even if no data is ready.
- It returns:
  - The number of bytes actually read/written (which could be zero), or
  - An error code like “would block”
- The process keeps running and can check later or try again - extra polling
- Example: A program calls `read()` → gets 0 bytes (no data yet) → does other work → tries again later.

I/O structure

- **Synchronous**
  - After I/O starts, control returns to user program only upon I/O completion
  - Wait instruction idles the CPU until the next interrupt
  - Wait loop (contention for memory access)
  - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- **Asynchronous**
  - After I/O starts, control returns to user program without waiting for I/O completion
  - **System call** allows application to request to the OS to allow user to wait for I/O completion
  - **Device-status table** contains entry for each I/O device indicating its type, address, and state

I/O structure

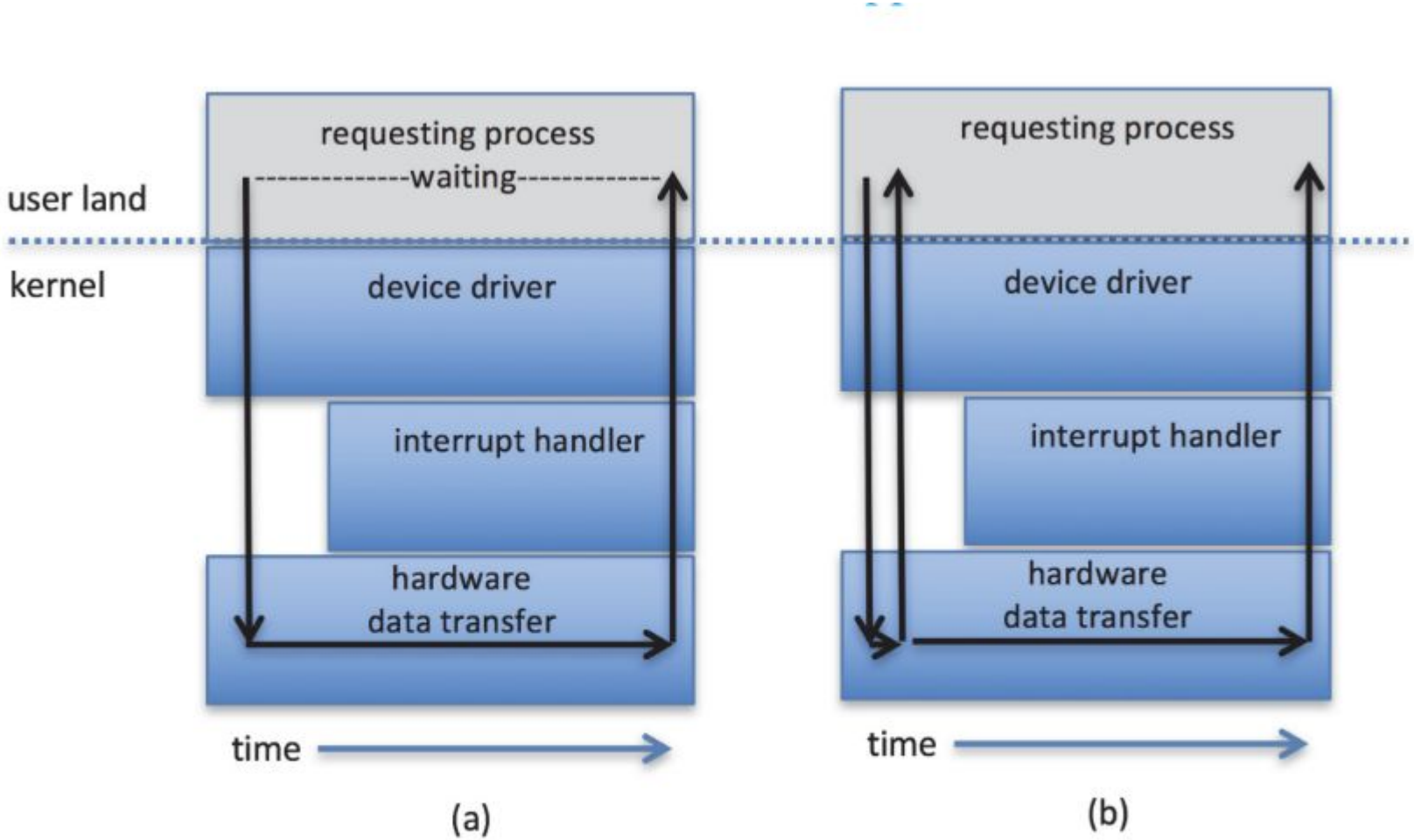


Figure 12.9 Two I/O methods: (a) synchronous and (b) asynchronous.

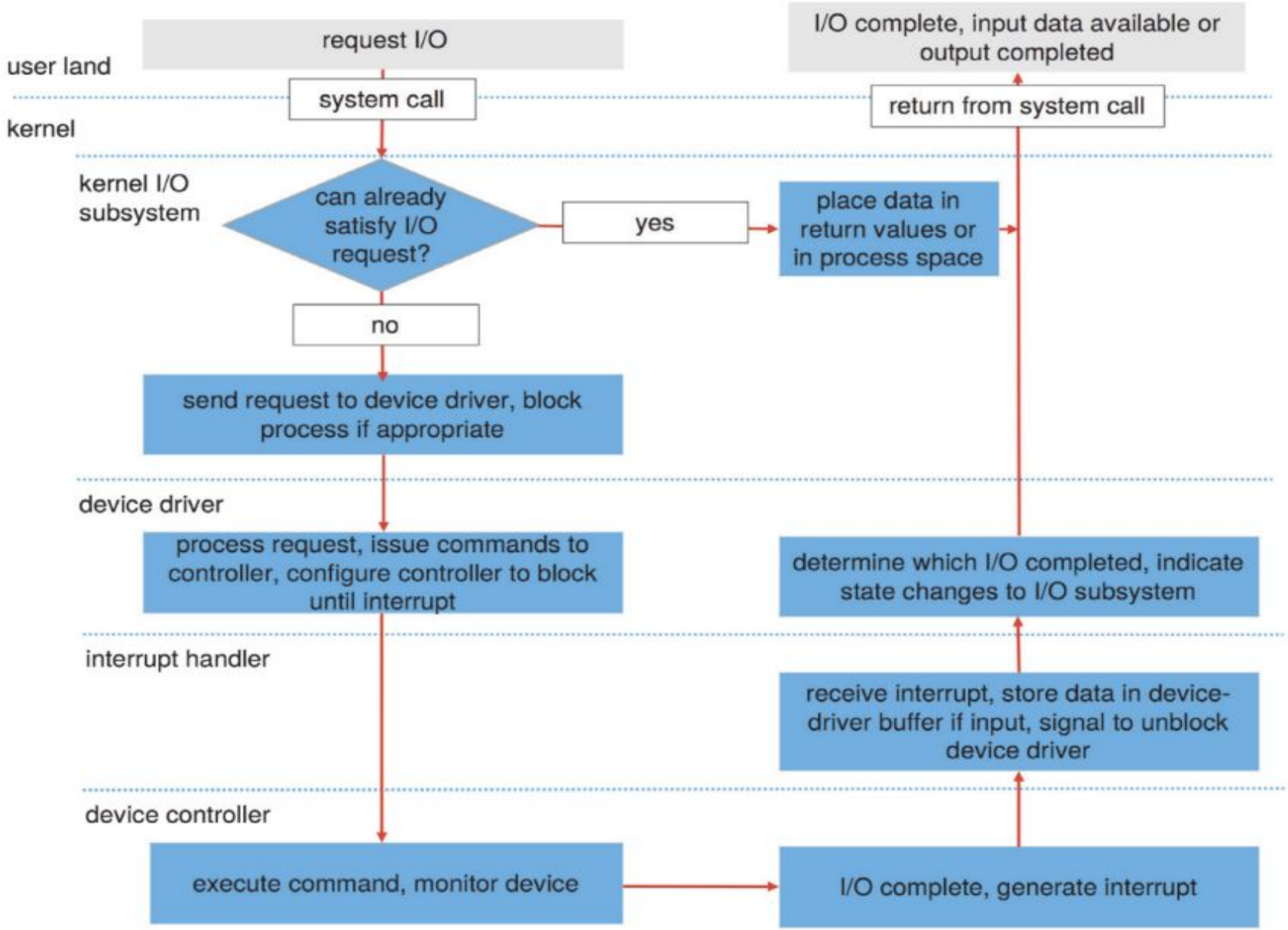


Figure 12.14: I/O request life-cycle

Outline

- I/O subsystem
- I/O devices
- Kernel data structures
  - Vectored I/O
  - Buffering
  - Other issues

Kernel data structures

The OS kernel maintains internal data structures to manage I/O efficiently and safely:

- Open file tables: Track which files are open by which processes.
- Network connection state: Sockets, ports, TCP/UDP states.
- Device status tables: For each device—type, address, busy/idle state, pending requests.
- Buffer descriptors: Manage memory buffers used for I/O (e.g., which are "dirty" and need writing to disk).

Vectored I/O

- Enable one system call to perform multiple I/O operations
  - E.g., Unix *readv() / writev()* accepts a vector of multiple buffers to read into or write from
- Also called **scatter-gather** method
- Better than multiple individual I/O calls
  - Decreases context switching and system call overhead

Buffering

Used to handle mismatches between devices speeds or transfer sizes:

- Single buffering: One system buffer per request.
- Double buffering: While CPU processes data from one buffer, I/O fills another buffer.
- Circular buffering: Multiple buffers in a ring—ideal for bursty I/O (e.g., audio, network).
- Details often dictated by device type: character devices buffer by line; network devices are very bursty

Other issues

- Caching: Fast memory copy of frequently used data (e.g., disk blocks in RAM)—critical for performance.
- Scheduling: Order of I/O requests (e.g., disk elevator algorithm) to reduce seek time.
- Spooling: Queue jobs for single-user devices (e.g., print spooler holds multiple print jobs).
- Error handling: Return error codes (e.g., “disk full”) and log failures.
- Protection: All I/O operations are privileged; user processes must use system calls to prevent crashes or security breaches.
- Performance: Affected by -
  - Context switches (from interrupts)
  - Data copying (user ↔ kernel ↔ device)
  - Driver efficiency

Summary

- I/O subsystem
  - Polling
  - Interrupts
  - Interrupt handling
    - Direct Memory Access (DMA)
- I/O devices
  - Device haracteristics
  - Blocking, non-blocking, asynchronous I/O
  - I/O structure
- Kernel data structures
  - Vectored I/O
  - Buffering
  - Other issues